Contender 11: Self-Height Diagonal Forest Closure of Kind-Polymorphic Reflection

A constructive large-number proposal for BusyLean

Research proposal for the BusyLean repository

October 12, 2026

Abstract

The current BusyLean construction, Contender 10, reflects over a kind-polymorphic System F_ω language with a unary oracle and then applies the resulting family twice at 42. This paper proposes a new closed, computable Lean natural number that strictly exceeds it by a formally simple but structurally strong closure operation. At each node of a self-indexing well-founded tree, we first perform a family-aware kind-polymorphic reflection, then build an iterated unary reflection tower whose height is the value of that first reflection plus one, and finally take a diagonal maximum. We iterate this node operator along the same W-language index that enumerates all trees representable by the repository's W-language, seed it with the construction of Contender 10, and apply the resulting family twice at 42.

The accompanying Lean modules contain the proposed definitions and a proof script for

`contender_10 < contender_11`.

The strict comparison is elementary once the construction is exposed: the new node operator explicitly dominates its child family, its bounded program enumeration, and its entire self-height reflection tower. No ordinal calibration is used in the formal inequality. Proof-theoretic discussion is therefore separated from the kernel-level claim and is stated only as a heuristic guide to scale. The number is computable in principle because every component is total and finitely bounded, although direct evaluation is wholly infeasible.

This archive was prepared against BusyLean commit `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`and Lean 4.34.0. The PDF and static source checks were completed in the artifact-building environment. That environment did not contain a Lean 4.34.0 executable, so the included replay script and axiom audit must still be run in a pinned checkout before the proposal is considered repository-accepted.

Contents

1 Executive summary **4**

2 Baseline and acceptance criteria **4**

2.1 Pinned repository state 4

2.2 Repository rules addressed by the proposal 5

3 Design objective **5**

4 The construction **6**
4.1 Family encoding . . . 6
4.2 Direct family-aware reflection . . . 6
4.3 Unary reflection towers . . . 7
4.4 The self-height node diagonal . . . 7
4.5 Forest closure . . . 8
4.6 Seed and final number . . . 8

5 Formal comparison **8**
5.1 Direct reflection bounds . . . 9
5.2 Strict growth of the tower . . . 9
5.3 What the node diagonal dominates . . . 10
5.4 Forest edges . . . 10
5.5 The seed path inside the self-index . . . 10
5.6 The central theorem . . . 11
5.7 Domination of W-definable trees . . . 11

6 Why the increase is substantial **11**
6.1 It closes the incumbent reflection under self-height iteration . . . 12
6.2 The reflection sees an entire child family . . . 12
6.3 The node operator is iterated along internal trees . . . 12
6.4 The final double application remains . . . 12

7 Computability and constructive legitimacy **12**
7.1 Why the number is computable in principle . . . 12
7.2 Why one should never try to evaluate it . . . 13
7.3 Axioms and proof irrelevance . . . 13

8 Proof-theoretic interpretation: heuristic only **13**

9 Lean artifact architecture **14**
9.1 Definition module . . . 14
9.2 Comparison module . . . 14
9.3 Audit and replay . . . 15

10 Reproducibility and verification status **15**
10.1 Applying the proposal . . . 15
10.2 Validation completed in this artifact build . . . 15
10.3 Expected audit outcome . . . 16

11 Risk analysis **16**
11.1 Formalization risks . . . 16
11.2 Metamathematical risks . . . 16

12 Further research questions **17**

13 Conclusion **17**

A Principal Lean definitions **18**

B Principal theorem signatures **19**

C Artifact inventory **19**

1 Executive summary

The proposed number is

$$C_{11} := F_{11}(F_{11}(42)),$$

where F_{11} is obtained by iterating a new node operator along the repository's self-indexing W-tree. The node operator has two nested forms of reflection:

1. A *family-aware* reflection $R_{\mathbf{f}}(n)$ presents a family $\mathbf{f}(i, m)$ to the kind-polymorphic object language through the repository's computable unary pairing and explicitly dominates all children $\mathbf{f}(i, n)$ with $i \leq n$.
2. A *self-height reflection tower* starts from the unary function $R_{\mathbf{f}}$ and iterates the existing operator `Fk.reflectFk` exactly $R_{\mathbf{f}}(n) + 1$ times before evaluating at n.
3. The node diagonal $D_{\mathbf{f}}(n)$ adds $n + 1$ to the maximum of the direct family-aware reflection and the self-height tower.
4. This node diagonal is recursively placed at every node of the W-tree `index`, whose branches cover all trees denoted by the repository's W-language.

The construction is designed so that the formal comparison is robust. The proof does not have to infer that a complicated evaluator is monotone, nor compare informal ordinal notations. It only uses explicit maximum bounds and three strict child steps already visible in the definition of `index`. The central chain is

$$C_{10} = \texttt{family10}(42) < F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

The increase is significant in a structural sense stronger than the bare inequality. At every node and input n, the new diagonal dominates:

- every visible child value $\mathbf{f}(i, n)$ for $i \leq n$;
- every accepted kind-polymorphic program in the depth-n enumeration, evaluated relative to the packed child family;
- every finite reflection tower of height at most $R_{\mathbf{f}}(n)$ over the direct family-aware reflection; and
- the input n itself.

Because $n < R_{\mathbf{f}}(n)$, this includes every tower whose height is bounded by the input, while the selected tower actually has the much larger height $R_{\mathbf{f}}(n) + 1$. That diagonal is then iterated through the W-language's own tree index.

2 Baseline and acceptance criteria

2.1 Pinned repository state

This proposal targets the repository state at commit `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`. At that commit the incumbent is Contender 10:

```
def family9 (n : Nat) : Nat := F9 (F9 n)
def F10 (n : Nat) : Nat := Fk.reflectFk family9 n
def contender_10 : Nat := F10 (F10 42)
```

The operator `Fk.reflectFk f n` is a finite maximum that exceeds both `f n` and every accepted closed program in a bounded enumeration of a kind-polymorphic System F_ω object language. Program values are computed by erasing types, normalizing with a total normalizer justified by the formal strong-normalization theorem, and decoding Church numerals.

Contender 9 already supplies another important ingredient. It exposes a binary family $\mathbf{f}(i, m)$ to a unary object-language oracle by the computable code

$$(i, m) \longmapsto 2^i(2m + 1),$$

whose inverse is implemented by `unpair2`. It also demonstrates forest reflection along the W-language's tree index. Contender 11 deliberately reuses these audited constructions rather than adding a new unverified evaluator.

2.2 Repository rules addressed by the proposal

The relevant acceptance conditions are summarized in table 1.

Table 1: BusyLean conditions and the corresponding design response.

Condition	Response in this proposal
Closed computable natural	`contender_11 : Nat` is built only from total recursive definitions, finite lists, finite maxima, and the previously certified total normalizer.
Formal strict comparison	`Comparison11.lean` contains a proof script for `contender_10_lt_contender_11`.
No nonconstructive stand-ins	The new definition uses no `noncomputable`, `partial`, `unsafe`, `opaque`, external primitive, custom axiom, or halting oracle.
Permitted axioms only	`Audit11.lean` prints axiom dependencies; `check_contender11.py` rejects anything beyond `propext` and `Quot.sound`.
No direct use of prior constant	The new definition uses `F10` as a construction, through `family10 n := F10 (F10 n)`, and never mentions `contender_10`. The closure audit checks all earlier contender constants.
Time budgets	The replay script treats `Comparison10` as precompiled history, then enforces 15 seconds for the new definition and 60 seconds for the proof phase.
Pinned toolchain	Replay instructions use Lean 4.34.0, matching the target repository.

The distinction in the fifth row is central to the contest. Reusing the *construction* of the predecessor as a seed is the accepted amplification pattern; naming the predecessor's closed constant and adding one would be a vacuous submission. Contender 11 follows the former pattern.

3 Design objective

A merely syntactic wrapper around C_{10} would formally win but would not be mathematically interesting. The objective is instead to close the strongest available repository mechanism under operations that are simultaneously:

1. constructive and directly executable in principle;

2. easy to compare by kernel-checkable inequalities;

3. qualitatively stronger than one additional fixed reflection step; and

4. economical enough to fit the repository's elaboration budget.

There are two natural amplification directions already present in BusyLean. First, `reflectFk` can be iterated as an operator on unary functions. Second, a family-valued reflection operator can be iterated along a well-founded tree, with the children made available to programs as an oracle family. A fixed-height tower or a single forest reflection would use only one of these directions. The present construction combines them and chooses the height of the unary tower by a diagonal value computed at the current tree node.

The key engineering constraint is that the proof of strict domination should not require a difficult semantic theorem about the growth of the evaluator. For this reason the node operator explicitly stores all quantities that must be bounded inside a maximum. The semantic machinery is needed for totality and for the meaning of program values, but the final comparison is arithmetic.

4 The construction

4.1 Family encoding

Let a family be a function

$$\mathbf{f} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \qquad \mathbf{f}(i, m) = \texttt{fs i m}.$$

The existing function `packFamily` turns it into a unary function by first decoding its input:

$$\widehat{\mathbf{f}}(q) = \mathbf{f}\big(\pi_1(\texttt{unpair2}(q)), \pi_2(\texttt{unpair2}(q))\big).$$

The repository proves

$$\widehat{\mathbf{f}}\big(2^i(2m+1)\big) = \mathbf{f}(i, m).$$

Thus a kind-polymorphic program with only a unary oracle can select any child index and any child input by ordinary natural-number computation.

4.2 Direct family-aware reflection

For a fixed input n, define the visible-child maximum

$$P_{\mathbf{f}}(n) = \max_{i \le n} \mathbf{f}(i, n)$$

and the bounded program maximum

$$M_{\mathbf{f}}(n) = \max\{\mathrm{Val}^{\widehat{\mathbf{f}}}_{\mathbf{f},n}(t) : t \in \mathrm{CTm}_{\le n}\}.$$

Here $\mathrm{CTm}_{\le n}$ is the finite list `Fk.ctmsUpTo n`, and Val is `Fk.fvalue`. The first new operator is

$$R_{\mathbf{f}}(n) = n + 1 + \max\big(P_{\mathbf{f}}(n), M_{\mathbf{f}}(n)\big). \tag{1}$$

Its Lean definition is:

```
def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum
      (Fk.fvalue (packFamily fs) n)
      (Fk.ctmsUpTo n))
```

The additive term is not cosmetic. It makes $n < R_{\mathbf{f}}(n)$ immediate even if both maxima happen to be zero. Likewise, every visible child and every bounded accepted program is strictly below $R_{\mathbf{f}}(n)$ rather than merely below or equal to it.

4.3 Unary reflection towers

Write $\mathcal{R}$ for the existing unary operator

$$\mathcal{R}(g)(n) = \texttt{Fk.reflectFk g n}.$$

Define its finite iterates by

$$\begin{aligned} \mathcal{T}_0(g) &= g, \\ \mathcal{T}_{k+1}(g) &= \mathcal{R}(\mathcal{T}_k(g)). \end{aligned}$$

The Lean definition returns a function after each recursion step:

```
def towerFk : Nat -> (Nat -> Nat) -> Nat -> Nat
  | 0, f     => f
  | Nat.succ k, f => Fk.reflectFk (towerFk k f)
```

The incumbent comparison theorem `Fk.oracle_lt_reflectFk` yields, for every g, k, n,

$$\mathcal{T}_k(g)(n) < \mathcal{T}_{k+1}(g)(n). \tag{2}$$

Consequently the tower is strictly increasing in its height at each fixed input.

4.4 The self-height node diagonal

The main diagonal choice is to let the direct reflection determine the number of subsequent unary reflection layers. Define

$$D_{\mathbf{f}}(n) = n + 1 + \max\left(R_{\mathbf{f}}(n), \mathcal{T}_{R_{\mathbf{f}}(n)+1}(R_{\mathbf{f}})(n)\right). \tag{3}$$

In Lean:

```
def diagonal11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (reflect11 fs n)
    (towerFk (reflect11 fs n + 1)
      (reflect11 fs) n)
```

This is a genuine self-height diagonal rather than an input-height tower. The number of layers is not n, nor a fixed constant, but the value of the direct family-aware reflection at n, plus one. Since $n < R_{\mathbf{f}}(n)$, it automatically subsumes every input-bounded finite tower. Since the tower component is stored verbatim in the maximum, the comparison does not require proving any further regularity property of $R_{\mathbf{f}}$ as a function of its input.

Remark 4.1. The tower is applied to the unary seed $R_{\mathbf{f}}$, not directly to one selected child. Thus the first layer already contains the complete visible-child and bounded-program maximum for the current family. Every later layer is a full kind-polymorphic reflection over the preceding layer.

4.5 Forest closure

Let W be the repository's countably branching well-founded tree type. The stage operator recursively places the node diagonal at every tree node:

$$S(\mathbf{s}, \mathrm{leaf})(n) = D_{\mathbf{s}}(n),$$
$$S(\mathbf{s}, \mathrm{node}(g))(n) = D_{\mathbf{f}_g}(n), \qquad \mathbf{f}_g(i, m) = S(\mathbf{s}, g(i))(m).$$

The Lean code is:

```
def stage11 (seed : W.Family) :
    BigNumber.WTree Nat -> Nat -> Nat
  | .leaf   => diagonal11 seed
  | .node g => diagonal11
      (fun i => stage11 seed (g i))
```

At a node, the family oracle is exactly the family of child stages. The prefix maximum makes each child with index at most the current input immediately smaller than the parent. The packed oracle simultaneously lets object-language programs call arbitrary children at arbitrary arguments through the unary code.

The tree used for the final family is the existing `index`. Its outer branches enumerate depths, and each depth branch joins the trees denoted by all W-language programs at that enumeration depth. The construction therefore closes the new node diagonal along the same internal ordinal resource already used successfully by Contenders 6 and 9.

4.6 Seed and final number

Turn Contender 10's construction into a family exactly as the preceding contenders do:

$$\mathrm{family10}(n) = F_{10}(F_{10}(n)).$$

In particular, $\mathrm{family10}(42) = C_{10}$ by definitional equality. Then set

$$F_{11}(n) = S(\mathrm{const}(\mathrm{family10}), \texttt{index})(n), \tag{4}$$
$$C_{11} = F_{11}(F_{11}(42)). \tag{5}$$

The complete final definitions are:

```
def family10 (n : Nat) : Nat := F10 (F10 n)

def F11 (n : Nat) : Nat :=
  stage11 (fun _ => family10) index n

def contender_11 : Nat := F11 (F11 42)
```

5 Formal comparison

This section gives the complete mathematical proof architecture mirrored by `Comparison11.lean`. The central point is that every nontrivial semantic quantity appears beneath an explicit maximum.

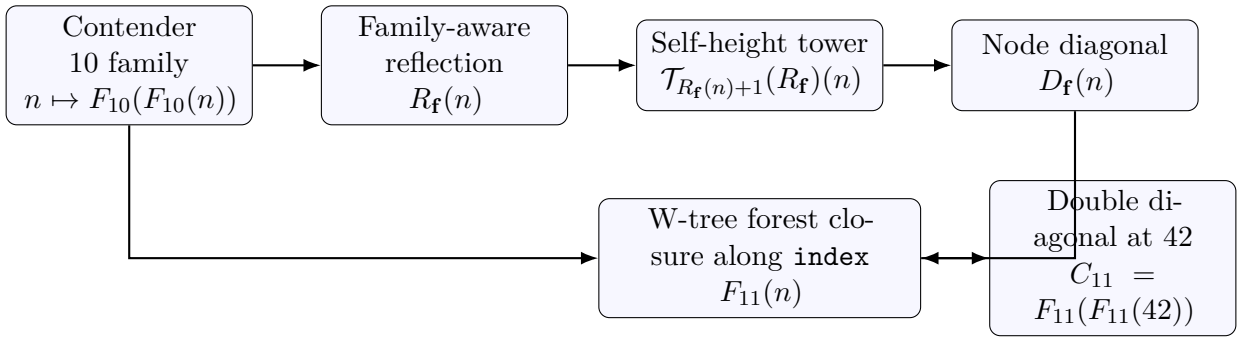


Figure 1: Construction pipeline. The tower height is itself the direct reflection value, and the entire node operator is then iterated along the self-indexing W-tree.

5.1 Direct reflection bounds

Lemma 5.1 (Visible children)**.** *For every family* $\mathbf{f}$*, input* n*, and index* $i \le n$*,*

$$\mathbf{f}(i, n) < R_{\mathbf{f}}(n).$$

Proof. By definition $\mathbf{f}(i, n) \le P_{\mathbf{f}}(n)$, and $P_{\mathbf{f}}(n)$ is one argument of the maximum in equation (1). The leading $n + 1$ turns the resulting weak bound into a strict one. □

Lemma 5.2 (Bounded programs)**.** *If* $t \in \mathrm{CTm}_{\le n}$*, then*

$$\mathrm{Val}^{\gg}_{\widehat{\mathbf{f}},n}(t) < R_{\mathbf{f}}(n).$$

Proof. The value is at most $M_{\mathbf{f}}(n)$ by the finite-maximum lemma already used throughout the repository, and $M_{\mathbf{f}}(n)$ is the second argument of the maximum in equation (1). □

Lemma 5.3 (Input)**.** *For all* $\mathbf{f}, n$*,* $n < R_{\mathbf{f}}(n)$*.*

Proof. Immediate from the additive $n + 1$ in equation (1). □

In Lean these are `oracle_lt_reflect11`, `fvalue_lt_reflect11`, and `input_lt_reflect11`.

5.2 Strict growth of the tower

Lemma 5.4 (One more reflection)**.** *For every unary* g *and every* k, n*,*

$$\mathcal{T}_k(g)(n) < \mathcal{T}_{k+1}(g)(n).$$

Proof. Unfold one step of the tower. The goal becomes

$$\mathcal{T}_k(g)(n) < \texttt{Fk.reflectFk}(\mathcal{T}_k(g), n),$$

which is exactly the incumbent theorem `Fk.oracle_lt_reflectFk`. □

Proposition 5.5 (Strict monotonicity in tower height)**.** *For fixed* g, n*, the function* $k \mapsto \mathcal{T}_k(g)(n)$ *is strictly increasing.*

Proof. Induct on the upper height. If $a < b + 1$, either $a = b$, in which case theorem 5.4 applies directly, or $a < b$, in which case combine the induction hypothesis with the final step from height b to $b + 1$. □

5.3 What the node diagonal dominates

Theorem 5.6 (Self-height tower closure)**.** *For every family* $\mathbf{f}$*, input* n*, and height* $k \le R_{\mathbf{f}}(n)$*,*

$$\mathcal{T}_k(R_{\mathbf{f}})(n) < D_{\mathbf{f}}(n).$$

Proof. The hypothesis gives $k < R_{\mathbf{f}}(n) + 1$. By theorem 5.5,

$$\mathcal{T}_k(R_{\mathbf{f}})(n) < \mathcal{T}_{R_{\mathbf{f}}(n)+1}(R_{\mathbf{f}})(n).$$

The latter is an explicit argument of the maximum in equation (3), hence is strictly below $D_{\mathbf{f}}(n)$. □

Corollary 5.7 (Input-bounded towers)**.** *If* $k \le n$*, then*

$$\mathcal{T}_k(R_{\mathbf{f}})(n) < D_{\mathbf{f}}(n).$$

Proof. By theorem 5.3, $n < R_{\mathbf{f}}(n)$, so $k \le R_{\mathbf{f}}(n)$ and theorem 5.6 applies. □

The direct reflection itself is another explicit argument of the maximum, so $R_{\mathbf{f}}(n) < D_{\mathbf{f}}(n)$. Combining this with theorems 5.1 to 5.3 gives:

Theorem 5.8 (Node domination)**.** *At every node and input* n*,* $D_{\mathbf{f}}(n)$ *strictly exceeds:*

1. *the input* n*;*
2. *every visible child* $\mathbf{f}(i, n)$ *with* $i \le n$*;*
3. *every accepted kind-polymorphic program in the depth-*n *enumeration, relative to the packed child family; and*
4. *every unary reflection tower over* $R_{\mathbf{f}}$ *of height at most* $R_{\mathbf{f}}(n)$*.*

5.4 Forest edges

Lemma 5.9 (Strict child edge)**.** *Let* $g : \mathbb{N} \to W$ *be the children of a tree node. For every* $i \le n$*,*

$$S(\mathbf{s}, g(i))(n) < S(\mathbf{s}, \mathrm{node}(g))(n).$$

Proof. At the parent, the oracle family is $\mathbf{f}_g(j, m) = S(\mathbf{s}, g(j))(m)$. Apply the visible-child part of theorem 5.8 with index i. □

The Lean theorem is `stage11_child_lt`. The companion theorem `input_lt_stage11` follows by cases on the tree and the input part of theorem 5.8.

5.5 The seed path inside the self-index

The proof that F_{11} dominates its seed uses a fixed three-edge path already present in the repository's comparison for Contender 9. The tree `index` is a node whose child 0 is `treesOfDepth 0`. That tree is a node whose child 0 is the tree denoted by the first depth-0 term; the repository proves that this tree is `leaf`. Hence, at every input n,

$$\begin{aligned}
\mathrm{family10}(n) &< S(\mathrm{const}(\mathrm{family10}), \mathrm{leaf})(n) \\
&< S(\mathrm{const}(\mathrm{family10}), \texttt{treesOfDepth 0})(n) \\
&< S(\mathrm{const}(\mathrm{family10}), \texttt{index})(n) \\
&= F_{11}(n).
\end{aligned}$$

The first inequality is node domination of the seed value; the next two are instances of theorem 5.9 with child index 0. This is formalized as `family10_lt_F11`.

5.6 The central theorem

Theorem 5.10 (Contender 11 beats Contender 10)**.**

$$C_{10} < C_{11}.$$

Equivalently, the Lean target is

$$\textit{contender_10} < \textit{contender_11}.$$

Proof. By definitional equality, $C_{10} = \text{family10}(42)$. The seed-path theorem gives

$$C_{10} < F_{11}(42).$$

The input theorem for the stage at input $F_{11}(42)$ gives

$$F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

Transitivity completes the proof. □

The corresponding Lean proof is deliberately short:

```
theorem contender_10_lt_contender_11 :
    contender_10 < contender_11 :=
  Nat.lt_trans (family10_lt_F11 42)
    (input_lt_stage11
      (fun _ => family10) index (F11 42))
```

5.7 Domination of W-definable trees

The same enumeration argument used for earlier forest contenders lifts unchanged.

Theorem 5.11 (Eventual domination of every represented tree)**.** *If a W-language term t occurs in the finite enumeration at depth k, then there exists N such that for every $n \geq N$,*

$$S(\text{const}(\text{family10}), \text{defTree}(t))(n) < F_{11}(n).$$

Proof. Choose the enumeration position i of t and take $N = k + i$. At any $n \geq N$, both $i \leq n$ and $k \leq n$. The represented tree is therefore a visible child of `treesOfDepth k`, and that depth tree is a visible child of `index`. Apply theorem 5.9 twice. □

The theorem appears as `defined_tree_eventually_lt11`.

6 Why the increase is substantial

The only unconditional numerical claim needed for acceptance is theorem 5.10. Nevertheless, the construction supplies a much richer margin than "predecessor plus one." Four independent facts support that assessment.

6.1 It closes the incumbent reflection under self-height iteration

Contender 10 applies `Fk.reflectFk` to the Contender 9 family and then uses an outer double application. At a Contender 11 node, the direct family-aware reflection value $R_{\mathbf{f}}(n)$ is itself used as the height of a new tower of `Fk.reflectFk` applications. This is not a fixed iterate chosen in the source, and not merely n iterations. The height is already larger than the input, every visible child, and every bounded accepted program value.

The formal theorem theorem 5.6 is therefore a useful intermediate result in its own right: one node of the new construction dominates every reflection tower whose height is at most the direct reflection value at that node.

6.2 The reflection sees an entire child family

A unary tower alone would amplify only one seed function. Here the direct seed at each node is a reflection relative to the packed family of all child stages. Programs in the object language may decode a dynamically computed child index and argument. Independently, the explicit prefix maximum makes the basic child inequality immune to whether a suitable selector program appears in the bounded enumeration.

This two-channel access is deliberate:

- *semantic access*: programs can combine and iterate child stages in ways determined at run time;
- *comparison access*: the proof can point directly to a finite prefix maximum and does not need to synthesize a selector program.

6.3 The node operator is iterated along internal trees

The self-height tower is not applied once at the root. It is the local operation at every node of `index`. The index is not a hand-written finite ordinal notation. Its branches range over all W-trees denoted by the repository's W-language, grouped by syntax depth. Thus the same strong local closure is propagated through an internally generated family of well-founded shapes.

6.4 The final double application remains

After building the forest-closed family F_{11}, the final number uses the same robust double diagonal pattern as Contenders 7–10:

$$F_{11}(F_{11}(42)).$$

The first value is already larger than C_{10}; the second call uses that value as the syntax budget, program-checker fuel, prefix length, tower input, and forest visibility bound throughout the construction.

7 Computability and constructive legitimacy

7.1 Why the number is computable in principle

Every operation in the definition is total:

1. `Fk.ctmsUpTo n` is a structurally recursive finite list.

2. `Fk.fvalue` uses a decidable bounded type checker and the existing total normalizer. The normalizer's termination argument is supplied by the formal strong-normalization development for the object language.

3. `BigNumber.maximum` and `W.prefixMax` traverse finite lists or finite prefixes.

4. `towerFk k f` is primitive recursion on the natural number k.

5. `stage11` is structural recursion on a well-founded W-tree, in the same form already accepted for earlier contenders.

6. `F11` and `contender_11` are ordinary compositions of these total functions.

There is no search for a halting witness, no maximum over all programs without a finite syntax bound, and no appeal to an uncomputable oracle. Given unlimited resources, reduction of `contender_11` reaches a numeral.

7.2 Why one should never try to evaluate it

Computable does not mean feasible. Even at modest inputs the syntax lists grow combinatorially, accepted terms can have enormous normal forms, and each reflection level embeds a maximum over all values from the preceding oracle. The self-height tower asks for a number of reflection layers equal to an already enormous reflected value. The W-tree closure then repeats that node operator recursively.

The repository's standard instruction therefore applies with extra force: check the definitions and proofs, but do not evaluate the contender. Kernel checking manipulates the compact syntax and proof terms; evaluation would try to realize the represented natural number.

7.3 Axioms and proof irrelevance

The new definitions add no axioms. They inherit the same metatheoretic foundation as Contender 10. The supplied audit prints the axiom dependencies of the new number and major comparison theorems and accepts only `propext` and `Quot.sound`, matching the repository policy. The audit also traverses the declaration closure of `contender_11` and rejects any occurrence of the closed constants `contender_10` through `contender_5`.

8 Proof-theoretic interpretation: heuristic only

The formal proof above deliberately avoids assigning an ordinal to the kind-polymorphic language or to the combined closure. That restraint is important. Exact proof-theoretic calibration of impredicative polymorphic systems, especially after oracle relativization and interaction with self-indexing W-trees, is subtle. A persuasive fast-growing-hierarchy slogan is not a substitute for a theorem.

Still, the construction has a clear qualitative reading. Let α denote, only schematically, the strength associated with one kind-polymorphic reflection step over a fixed oracle. Then:

- one application of `reflectFk` behaves like one strong reflection increment over the oracle;
- a tower of height k iterates that increment k times;

- choosing $k = R_{\mathbf{f}}(n) + 1$ diagonalizes the iteration length against a value already produced by the reflected system; and
- forest closure repeats this diagonal operation along W-language-definable well-founded trees.

This suggests a jump far beyond a fixed finite stack of Contender 10-style reflections. The archive does *not* claim an exact ordinal identity, a complete domination theorem over all functions definable in some external foundational theory, or a separation result between proof-theoretic systems. Those would require a separate metamathematical development.

The strongest rigorous growth statement currently supplied is internal and finite: theorem 5.6, together with forest child domination and theorem 5.11. These are already enough to show that the proposal is not merely a constant-size arithmetic wrapper.

9 Lean artifact architecture

9.1 Definition module

`BusyLean/Contender11.lean` contains only the definitions required to name the number:

Declaration	Purpose
`reflect11`	Family-aware bounded kind-polymorphic reflection.
`towerFk`	Finite iteration of the existing unary reflection operator.
`diagonal11`	Maximum of the direct reflection and its self-height tower, plus the input guard.
`stage11`	W-tree recursion placing the diagonal at every node.
`family10`	Contender 10's construction viewed as an input family.
`F11`	Forest closure along `index`.
`contender_11`	The final closed natural $F_{11}(F_{11}(42))$.

Keeping comparisons out of the definition module is consistent with the repository's timed build discipline: the definition phase can be measured separately from theorem elaboration.

9.2 Comparison module

`BusyLean/Comparison11.lean` is organized in four layers:

1. arithmetic bounds for `reflect11`;
2. strict monotonicity of `towerFk`;
3. node and forest domination theorems; and
4. seed, contender, and definable-tree comparisons.

The proof intentionally reuses the public comparison interface of Contender 10, not the internal normalization proof. In particular, `towerFk_step_lt` is one line after unfolding because `Fk.oracle_lt_reflectFk` already states exactly the needed inequality.

9.3 Audit and replay

`test/Audit11.lean` contains:

- `#print axioms` commands for the number and major theorems;
- a transitive declaration-closure command that rejects all prior closed contender constants; and
- `#check` commands recording the central signatures.

`scripts/check_contender11.py` performs the repository-style clean build, enforces time budgets, runs the audit, parses its output, and rejects forbidden axioms. The patch also adds the two modules to `BusyLean.lean`.

10 Reproducibility and verification status

10.1 Applying the proposal

Starting from the pinned commit, the intended replay is:

```
git checkout 4c4b03d0e2003c479e3bee90a21c8bea13f66c40
git apply patches/contender11.patch
python3 scripts/check_contender11.py
```

Equivalently, copy the files from the archive's `lean/` directory into the matching repository paths and add the two imports to `BusyLean.lean`.

10.2 Validation completed in this artifact build

The following checks were run while producing the archive:

1. the LaTeX article was compiled to PDF;
2. the PDF was rendered page by page and visually inspected for clipping, overlap, black boxes, and broken glyphs;
3. the Python replay script was syntax-checked;
4. a static policy scanner checked the new definition for forbidden Lean features and checked that executable code does not reference a prior closed contender constant;
5. the integration patch was applied with `git apply -check` to a reconstructed pinned-file skeleton; and
6. SHA-256 hashes were generated for the deliverables.

Verification status 10.1. A Lean 4.34.0 executable was not available in the artifact-building container, and the official binary could not be materialized through the available file transport. Therefore this report does not claim that the new modules were kernel-replayed here. The proof scripts were reviewed against the pinned source interfaces and are accompanied by an exact replay command. Repository acceptance should be conditional on a successful run of `python3 scripts/check_contender11.py` in the pinned checkout.

This limitation affects confidence in elaboration details, not the mathematical comparison argument: the latter is given explicitly in section 5 and uses only the stated incumbent lemmas. Possible replay failures would most likely be local syntactic or inference issues and should not require changing the construction.

10.3 Expected audit outcome

A successful replay should establish all of the following:

- the definition module elaborates within the 15-second budget;
- the comparison module and complete library elaborate within the 60-second proof budget;
- no axiom outside `propext` and `Quot.sound` appears in the audited declarations;
- `contender_11`'s declaration closure avoids `contender_10`, `contender_9`, and all earlier contender constants; and
- the theorem `contender_10_lt_contender_11` is accepted by the Lean kernel.

11 Risk analysis

11.1 Formalization risks

The proposal is intentionally conservative in its use of new Lean machinery. The main residual risks are:

1. **Elaboration of the function-valued recursion.** The return type of `towerFk` is curried. This is standard structural recursion, but the exact equation reduction should be confirmed on Lean 4.34.0.
2. **Strict-monotonicity proof inference.** The induction in `towerFk_strictMono` uses only natural arithmetic and the one-step theorem. If elaboration is sensitive, it can be replaced by an explicitly quantified two-height lemma without changing any definition.
3. **Timing.** The new definition is syntactically small and reuses precompiled Contender 10 modules, so it is expected to fit comfortably. Only an actual clean build can certify the repository's machine-relative timing rule.
4. **Axiom reporting through inherited normalization proofs.** The new number inherits the same proof-carrying normalizer as Contender 10. The audit is included precisely to ensure the inherited closure stays within the repository's allowed pair of axioms.

None of these risks concerns termination of the mathematical construction or validity of the inequality argument.

11.2 Metamathematical risks

No exact proof-theoretic ordinal is claimed. A future calibration could reveal that some apparently different amplification mechanisms collapse under an appropriate equivalence. Such a result would not invalidate the strict numerical theorem or the internal tower-domination theorems. It would only refine the informal account of how large the margin is.

12 Further research questions

The construction suggests several concrete next projects.

1. **Kernel replay and proof minimization.** Run the pinned audit, remove any unnecessary uses of `omega`, and seek an axiom-free arithmetic comparison layer if the inherited definitions permit it.

2. **Uniform tower domination.** Strengthen `fixedTower_lt_diagonal11` from a pointwise height bound to domination of a class of height functions $h(n) \leq R_{\mathbf{f}}(n)$, with a reusable Lean API.

3. **Internal tower coding.** Construct explicit kind-polymorphic terms that realize selected reflection-tower patterns, clarifying how much of the external tower can be represented inside the object language.

4. **Self-index from the stronger language.** The current tree index is inherited from the W-language because it has a direct Lean set-theoretic interpretation. Develop a safe syntax of well-founded codes generated by the kind-polymorphic language itself, together with a total decoding theorem.

5. **Predicative kind-universe ladder.** Implement a stratified sequence of kind universes and diagonalize over the number of levels, as anticipated in the repository roadmap. Compare that language-level jump with the present external self-height closure.

6. **Large elimination.** Build a realizability model for a carefully chosen dependently typed object language with natural-number elimination into types, retaining a total executable normalizer and a sound bounded checker.

7. **Formal growth separations.** Identify restricted fragments for which one can prove that a self-height tower eventually dominates every fixed tower, rather than relying only on the pointwise theorem supplied here.

8. **Resource-certified checking.** Extend the audit so that the timing report, toolchain hash, source commit, and axiom output are emitted as a machine-readable attestation committed with each contender.

9. **Alternative diagonal heights.** Compare $R_{\mathbf{f}}(n) + 1$ with heights such as $D_{\mathbf{f}}(n)$ truncated through a structurally decreasing recursion, or heights generated by a separate W-recursive rank, while preserving obvious totality and simple comparison proofs.

10. **Proof-theoretic calibration.** Develop a rigorous ordinal or reflection-theoretic analysis of kind-polymorphic F_ω with oracles, then analyze finite iteration and W-tree closure as separate operations.

13 Conclusion

Contender 11 is a conservative formal extension and an aggressive growth extension. It does not add a new evaluator, axiom, or unbounded search. Instead, it composes three mechanisms already justified in BusyLean:

1. kind-polymorphic reflection over a total object language;

2. computable packing of an oracle family; and

3. forest reflection along a self-indexing W-tree.

The new contribution is the self-height diagonal between the first and third mechanisms. At each node, the result of direct family-aware reflection chooses the height of an entire subsequent reflection tower. Explicit maxima make the key theorem elementary, while the recursive placement of that diagonal along `index` supplies a substantial structural margin.

The resulting closed natural

$$\boxed{C_{11} = F_{11}(F_{11}(42))}$$

is computable in principle and is accompanied by a short formal comparison script, an axiom/dependency audit, a timed replay script, an integration patch, and this detailed argument. The remaining acceptance step is mechanical but essential: replay the archive under Lean 4.34.0 at the pinned repository commit.

A Principal Lean definitions

For review convenience, the core definition module is reproduced below in a compact form. The authoritative source is `lean/BusyLean/Contender11.lean` in the archive.

```
import BusyLean.Contender10

namespace BusyLean

def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum
      (Fk.fvalue (packFamily fs) n)
      (Fk.ctmsUpTo n))

def towerFk : Nat -> (Nat -> Nat) -> Nat -> Nat
  | 0, f     => f
  | Nat.succ k, f => Fk.reflectFk (towerFk k f)

def diagonal11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (reflect11 fs n)
    (towerFk (reflect11 fs n + 1)
      (reflect11 fs) n)

def stage11 (seed : W.Family) :
    BigNumber.WTree Nat -> Nat -> Nat
  | .leaf   => diagonal11 seed
  | .node g => diagonal11
      (fun i => stage11 seed (g i))

def family10 (n : Nat) : Nat := F10 (F10 n)

def F11 (n : Nat) : Nat :=
  stage11 (fun _ => family10) index n
```

```
def contender_11 : Nat := F11 (F11 42)

end BusyLean
```

B Principal theorem signatures

```
theorem oracle_lt_reflect11
  (fs : W.Family) {i n : Nat} (hi : i <= n) :
  fs i n < reflect11 fs n

theorem fvalue_lt_reflect11
  (fs : W.Family) {n : Nat} {t : Fk.CTm}
  (ht : t in Fk.ctmsUpTo n) :
  Fk.fvalue (packFamily fs) n t < reflect11 fs n

theorem towerFk_step_lt
  (f : Nat -> Nat) (k n : Nat) :
  towerFk k f n < towerFk (k + 1) f n

theorem fixedTower_lt_diagonal11
  (fs : W.Family) {k n : Nat}
  (hk : k <= reflect11 fs n) :
  towerFk k (reflect11 fs) n < diagonal11 fs n

theorem stage11_child_lt
  (seed : W.Family)
  (g : Nat -> BigNumber.WTree Nat)
  {i n : Nat} (hi : i <= n) :
  stage11 seed (g i) n < stage11 seed (.node g) n

theorem family10_lt_F11 (n : Nat) :
  family10 n < F11 n

theorem contender_10_lt_contender_11 :
  contender_10 < contender_11
```

The actual source uses Lean's membership symbol in two signatures; the ASCII word `in` above is expository pseudocode, not a replacement for the archive's authoritative module.

C Artifact inventory

Path	Purpose
`article/contender11.tex`	This paper's LaTeX source.
`article/contender11.pdf`	Compiled and rendered paper.
`lean/BusyLean/Contender11.lean`	New closed-number definitions.
`lean/BusyLean/Comparison11.lean`	Strict comparison and structural domination theorems.
`lean/test/Audit11.lean`	Axiom and dependency closure audit.

Path	Purpose
`lean/scripts/check_contender11.py`	Clean-build, timing, and audit driver.
`patches/contender11.patch`	Patch against the pinned BusyLean commit.
`notes/FORMAL_CLAIMS.md`	Claim-by-claim map from mathematics to declarations.
`notes/REPRODUCE.md`	Exact integration and replay instructions.
`validation/static_validation.txt`	Results of artifact-level static checks.
`validation/pdf_inspection.txt`	PDF render and visual inspection record.
`MANIFEST.sha256`	SHA-256 inventory of final files.

References

[1] V. Reshetnikov and contributors, *BusyLean: name the biggest number, in Lean*, GitHub repository, commit `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`, 2026.

[2] J.-Y. Girard, *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*, Thèse d'État, Université Paris VII, 1972.

[3] J. C. Reynolds, Towards a theory of type structure, in *Programming Symposium*, Lecture Notes in Computer Science 19, Springer, 1974, pp. 408–425.

[4] T. Coquand and G. Huet, The calculus of constructions, *Information and Computation* 76 (1988), 95–120.

[5] W. A. Howard, Ordinal analysis of terms of finite type, *Journal of Symbolic Logic* 45 (1980), 493–504.